Chapter 5 CPU Scheduling

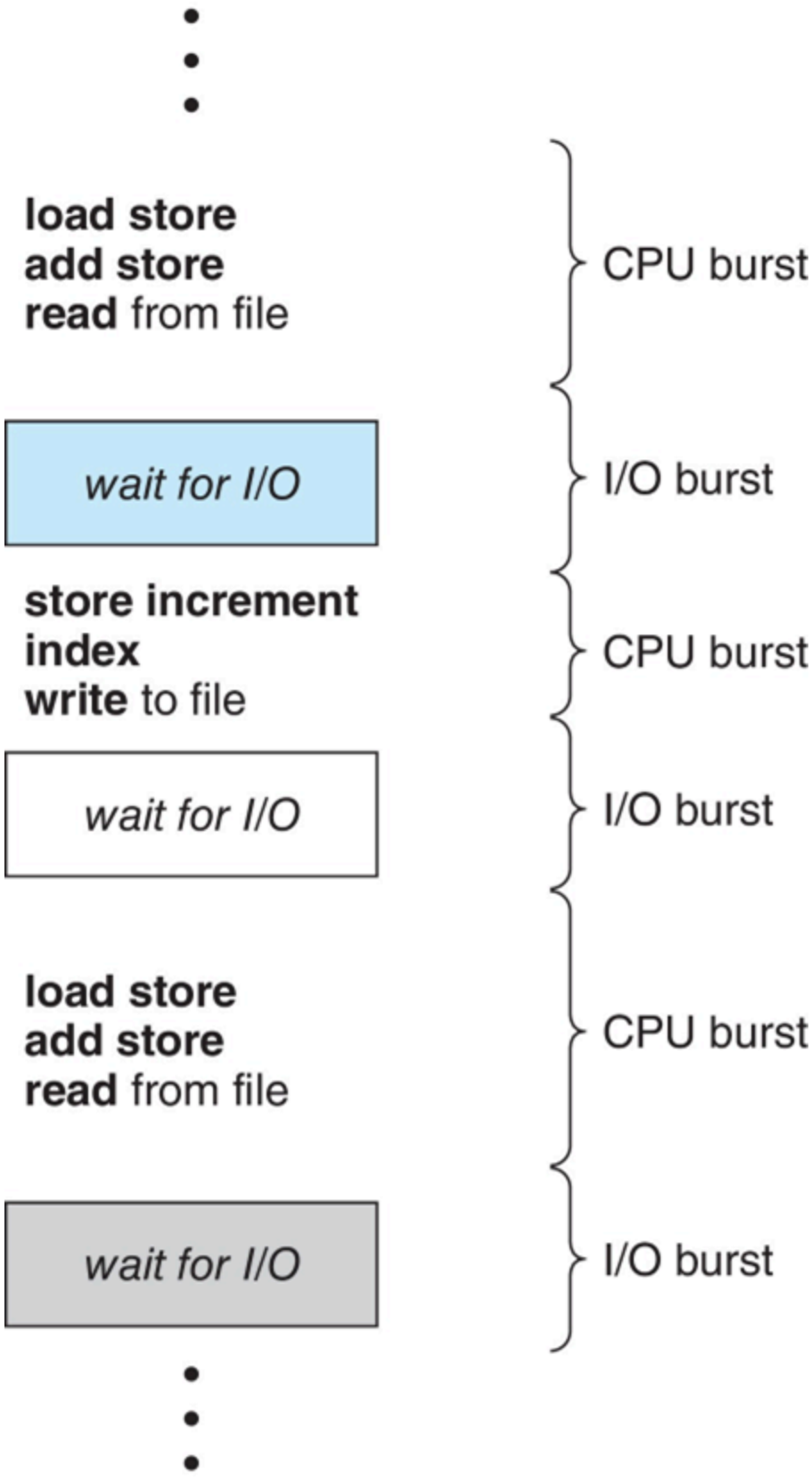
Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU scheduling
- Operating systems examples
- Algorithm evaluation

Objectives

- Describe various CPU scheduling algorithms
- Assess CPU scheduling algorithms based on scheduling criteria
- Explain the issues related to multiprocessor and multicore scheduling
- Describe various real-time scheduling algorithms
- Describe the scheduling algorithms used in the Windows, Linux, and Solaris operating systems.
- Apply modeling and simulations to evaluate CPU scheduling algorithms

Basic Concepts

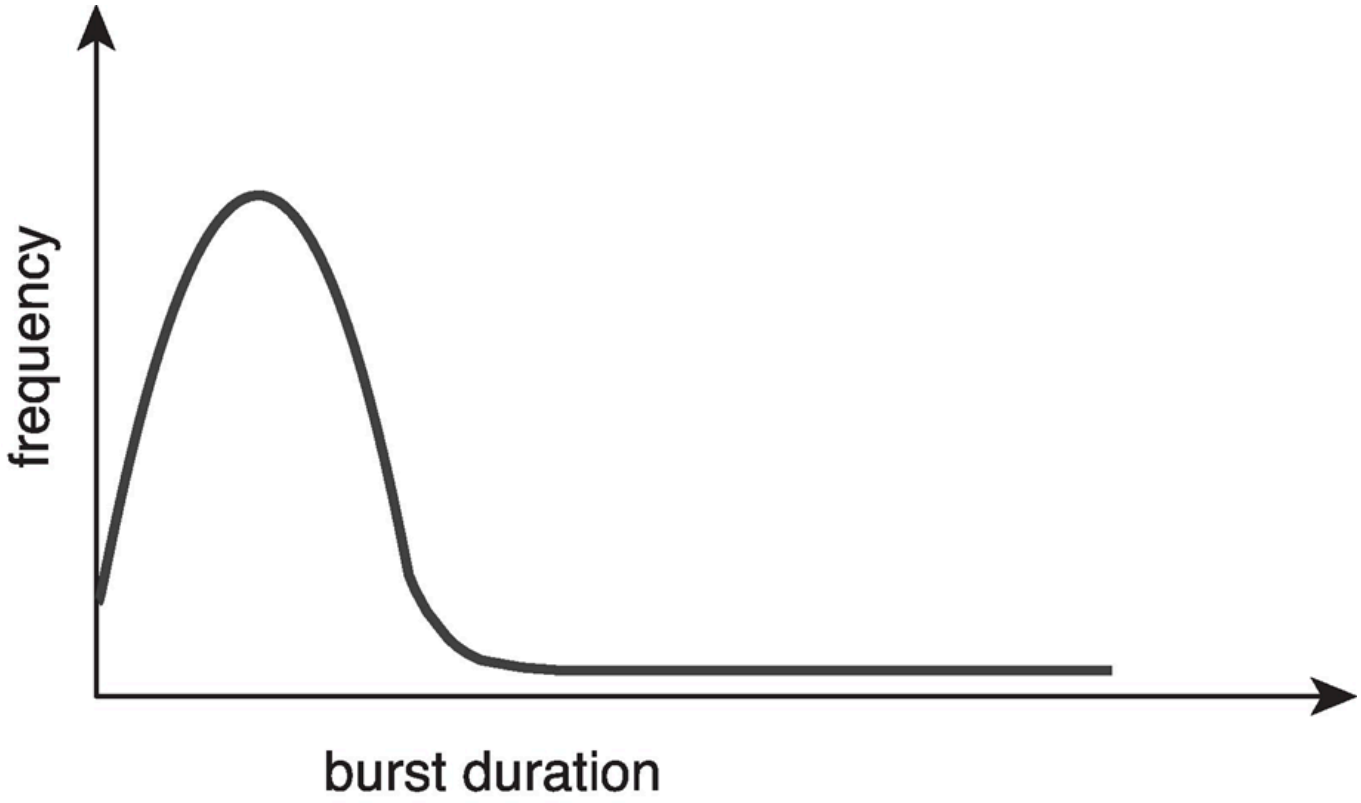


CPU-I/O burst cycle -- Process execution consists of a cycle of CPU execution and I/O wait. (CPU burst followed by I/O burst)

CPU burst distribution is of main concern. Maximum CPU utilization obtained with multiprogramming.

Histogram of CPU-Burst Times

Large number of short bursts, small number of longer bursts.



CPU Scheduler

The CPU scheduler selects from among the processes in queue and allocates a CPU core to one of them.

CPU scheduling decisions may take place when a process

- switches from running to waiting state
- switches from running to ready state
- switches from waiting to ready
- terminates

For 1 and 4, no choices in terms of scheduling. A new process must be selected for execution. For 2 and 3, there is a choice.

Preemptive and Nonpreemptive Scheduling

For above 1, 4, the scheduling scheme is nonpreemptive(not exclusive) otherwise it is preemptive. For nonpreemptive scheduling, the CPU keeps used until the process releases it either by terminating or by switching to the waiting state(dominated by processes). For

preemptive scheduling, OS can use schedule CPU forcefully(most modern OSs use preemptive scheduling)

Preemptive Scheduling and Race Conditions

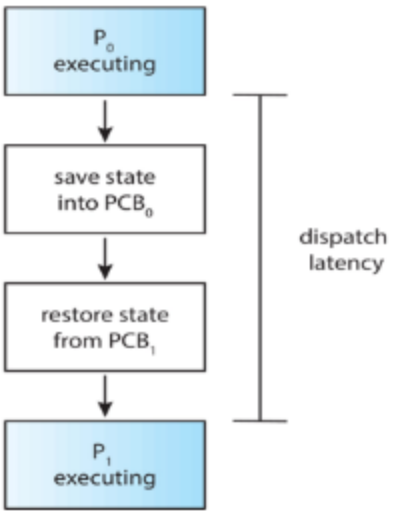
Preemptive scheduling can result in race conditions when data are shared among several processes.

Dispatcher

Dispatcher module controls of CPU to the process selected by the CPU scheduler.

- switch context
- switching to user mode
- jump to the proper location in the user program to restart the program

The dispatch latency is the time takes for the dispatcher to stop one process and start another running.



Scheduling Criteria

- CPU utilization: keep the CPU as busy as possible
- Throughput: the number of completed process per time unit.
- Turnaround time: amount of time to execute a particular process
- Waiting time: amount of time a process has been waiting in the ready queue
- Response time: amount of time it takes from a request was submitted until the first response is produced.

Scheduling Algorithm Optimization Criteria

- Max CPU utilization

- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

First-Come, First-Served (FCFS) Scheduling

The scheduling is simple but average waiting time is long. It is not good for short process(waiting time is long), called Convey effect: short process behind long process.

Shortest-Job-First (SJF) Scheduling

It associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time. It is optimal, giving minimum average waiting time for a set of processes. Preemptive version called shortest-remaining-time-first. However, the length is often estimated. It can use exponential averaging.

Shortest Remaining Time First(SRTF) Scheduling

It is preemptive version of SJN. Whenever a new process arrives in the ready queue, the scheduling is redone using SJF scheduling.

When the arriving time of processes are not the same, SRTF is better than SJF.

Round Robin Scheduling

Each process gets a small unit of CPU time, q, as time slice, usually 10-100 milliseconds. After the time, the process is preempted and added to the end of the ready queue. No process waits more than $(n-1)q$ time units. Timer interrupts every quantum to schedule next process.
When q is large, FCFS is better. When q is small, RR is better.

- q must be large with respect to context switch otherwise overhead is too high.
- q should cover approximate 80% CPU bursts to balance the turnaround time and the response.
  Typically, higher average turnaround than SJF but better than response.

Priority Scheduling

A priority number(integer) is associated with each process. The CPU is allocated to the process with the highest priority. (Smallest integer is highest priority)

SJF is priority scheduling where priority is the inverse of predicted next CPU burst time.

To solve the problem that low priority processes may never execute(stravation), the priority of the process will increase as time progresses(aging).

Priority can combine with round robin. Run the process with the highest priority. Processes with the same priority run round-robin.

Multilevel Queue

The ready queue consists of multiple queues to achieve the multilevel queue.
Multilevel queue scheduler defined with the following parameters:

- Number of queues
- Scheduling algorithms for each queue
- Method used to determine which queue a process will enter when that process needs service
- Scheduling among queues.

Similar to priority scheduling, we set priority for different queues. Scheduler will schedule the process in the highest-priority queue.

Multilevel Feedback Queue

A process can move between the various queues.
The queue is defined by the following parameters:

- Number of queues
- Scheduling algorithms for each queue
- Method used to determine when to upgrade a process
- Method used to determine when to demote a process
- Method used to determine which queue a process will enter when the process needs service
  Aging can also be implemented using multilevel feedback queue.

Thread Scheduling

- Distinction between user-level and kernel-level threads.
- When threads supported, threads scheduled rather than processes

Two types of thread scheduling:

- Thread library schedules user-level threads to run on LWP(Light-Weight process) by many-to-one and many-to-many models, known as process-contention scope(PCS) since scheduling competition is within the process.

- Kernel thread scheduled onto available CPU is system-contention scope(SCS).

Pthread Scheduling

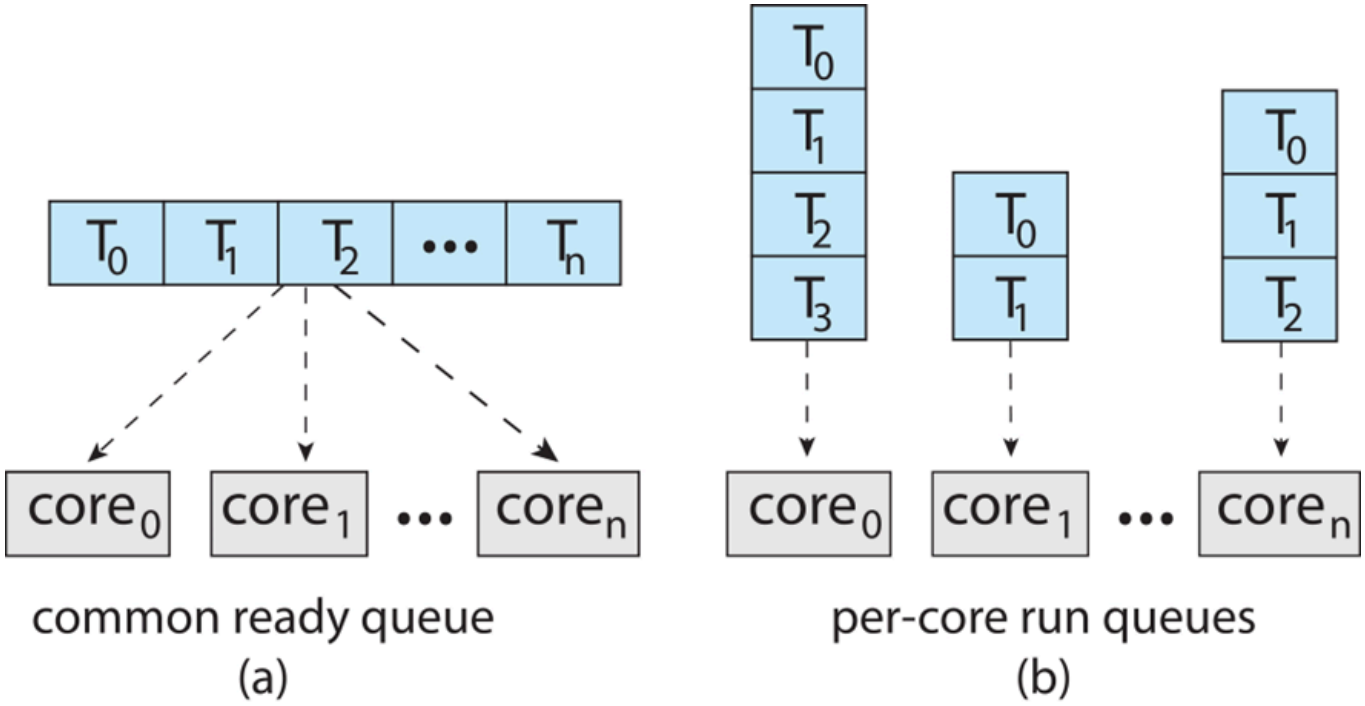
- API allows specifying either PCS or SCS during thread creation. (PTHREAD_SCOPE_PROCESS and PTHREAD_SCOPE_SYSTEM)
- Some OSs only allow PTHREAD_SCOPE_SYSTEM(Linux, MacOS)

Multiple-Processor Scheduling

Multiprocess may be any one of the following architectures

- Multicore CPUs
- Multithreaded cores
- NUMA systems
- Heterogeneous multiprocessing

Symmetric multiprocessing (SMP) is where each processor is self scheduling. All threads may be in a common ready queue. Each processor may have their own private queue of threads.

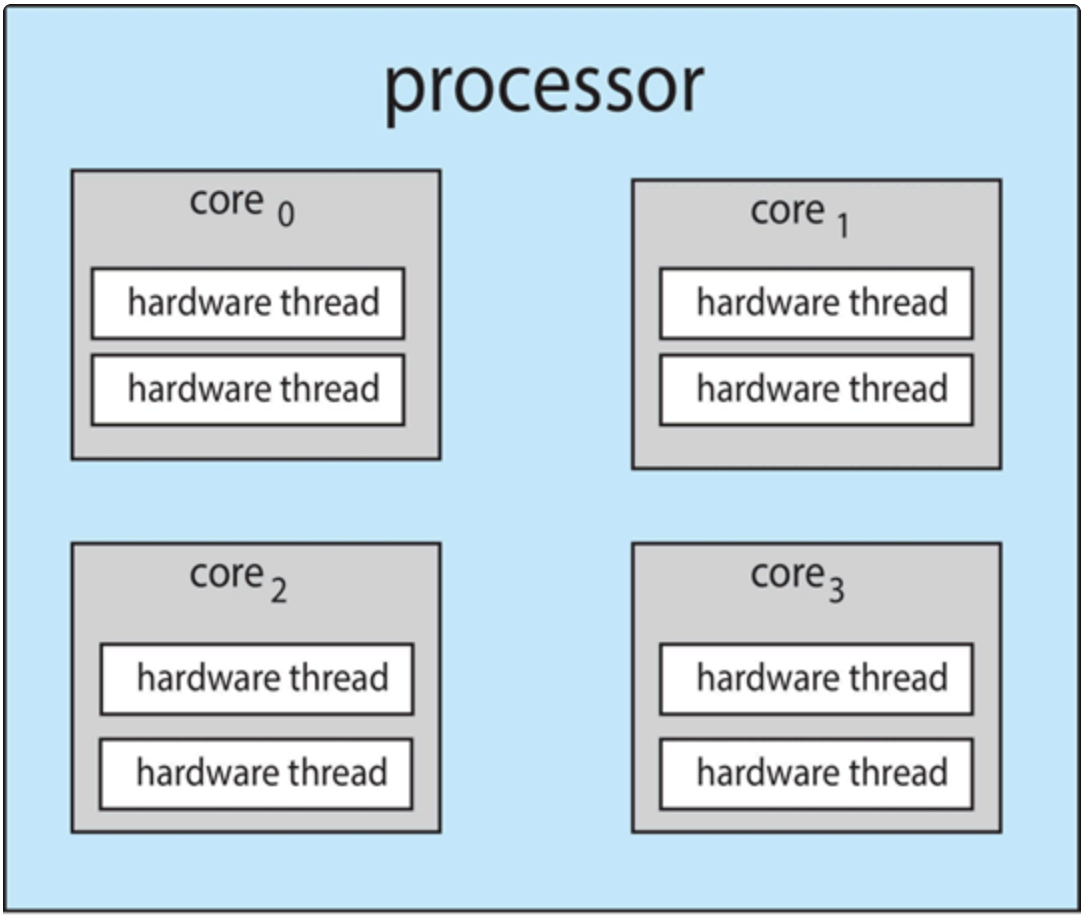


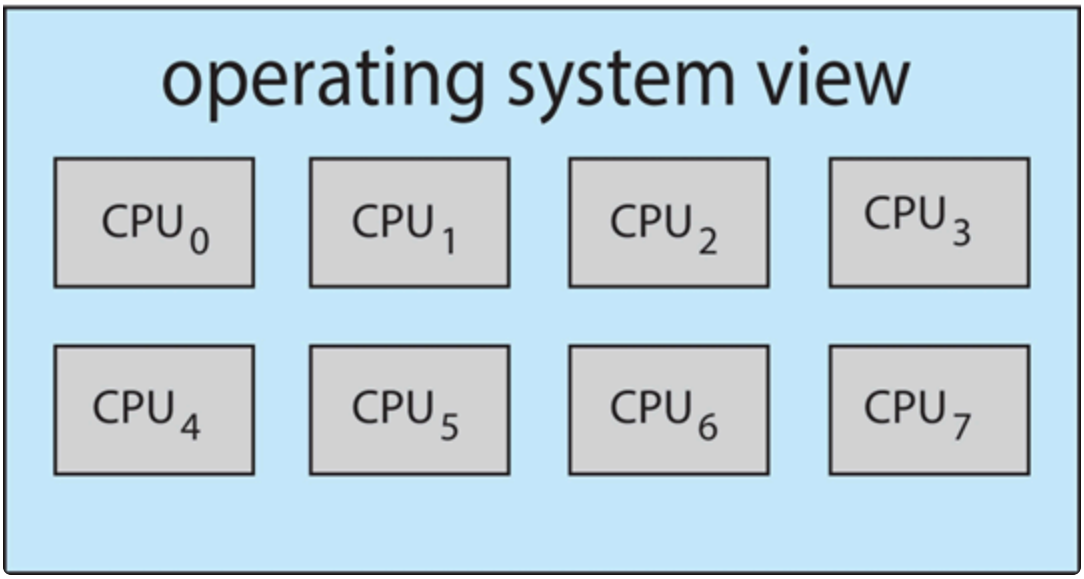
Multicore CPUs

Recent trend to place multiple processor cores on same physical chip. It is faster and consumes less power. Meanwhile, multiple threads per core takes advantage of memory stall to make progress on another thread while memory retrieve happens.(switch threads when the memory pause)

Multithreaded Cores

Chip-multithreading (CMT) assigns each core multiple hardware threads(known as hyperthreading). (OS sees multiple logical processors)





Two levels of scheduling:

- OS decides which software thread to run on a logical CPU
- Each core decides which hardware thread to run on the physical core

Symmetric multiprocessing need to keep all CPUs loaded for efficiency by load balancing.

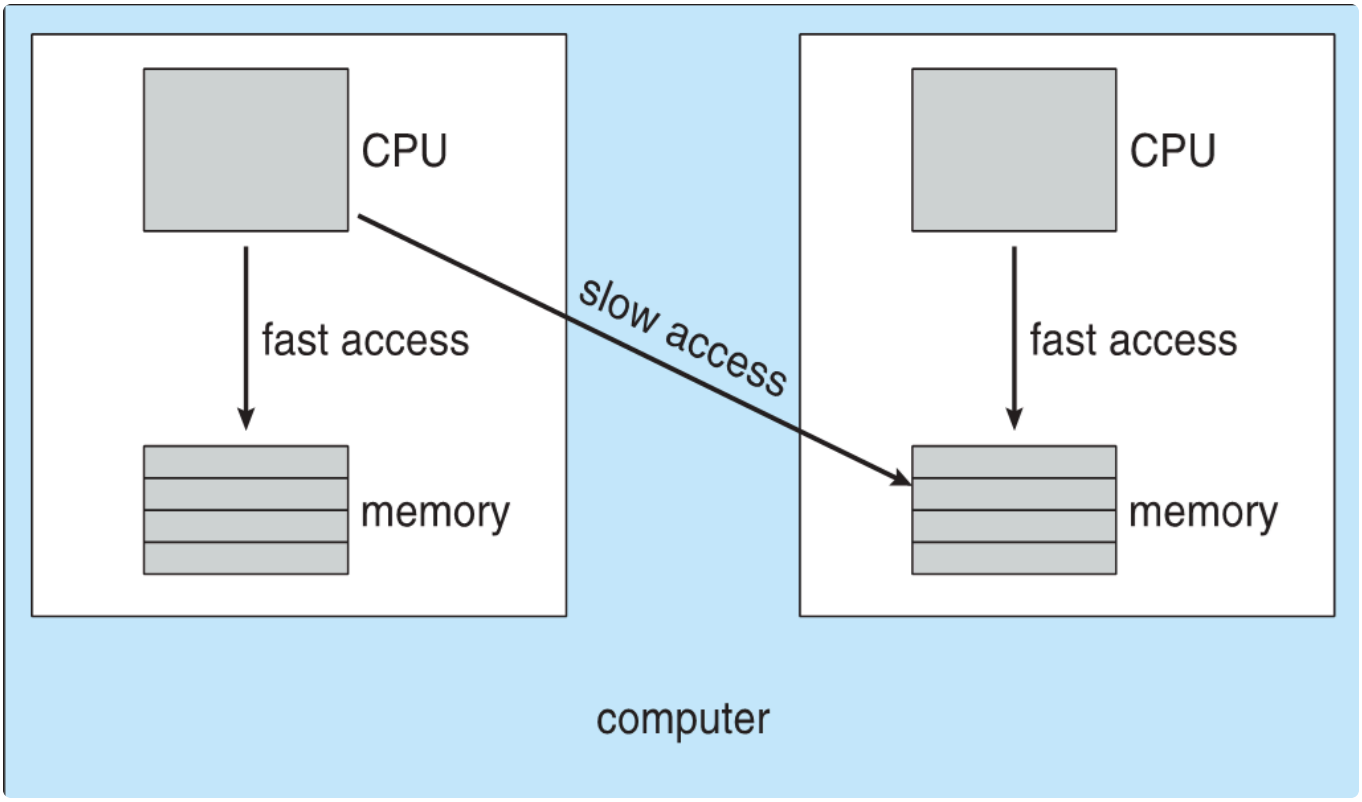
- Push migration: periodic task checks load on each processor and if found pushes task from overloaded CPU to other CPUs.
- Pull migration: idle processors pulls waiting task from busy processor

When a thread has been running on one processor, the cache stores the data generated by that thread, called processor affinity. Load balancing may affect processor affinity due to switching processors.

- Soft affinity: OS attempts to keep a thread running on the same processor but no guarantees
- Hard affinity: Allows a process to specify a set of processes it may run on.

NUMA and CPU Scheduling

NUMA-aware OS will assign memory closes to the CPU the thread is running on.



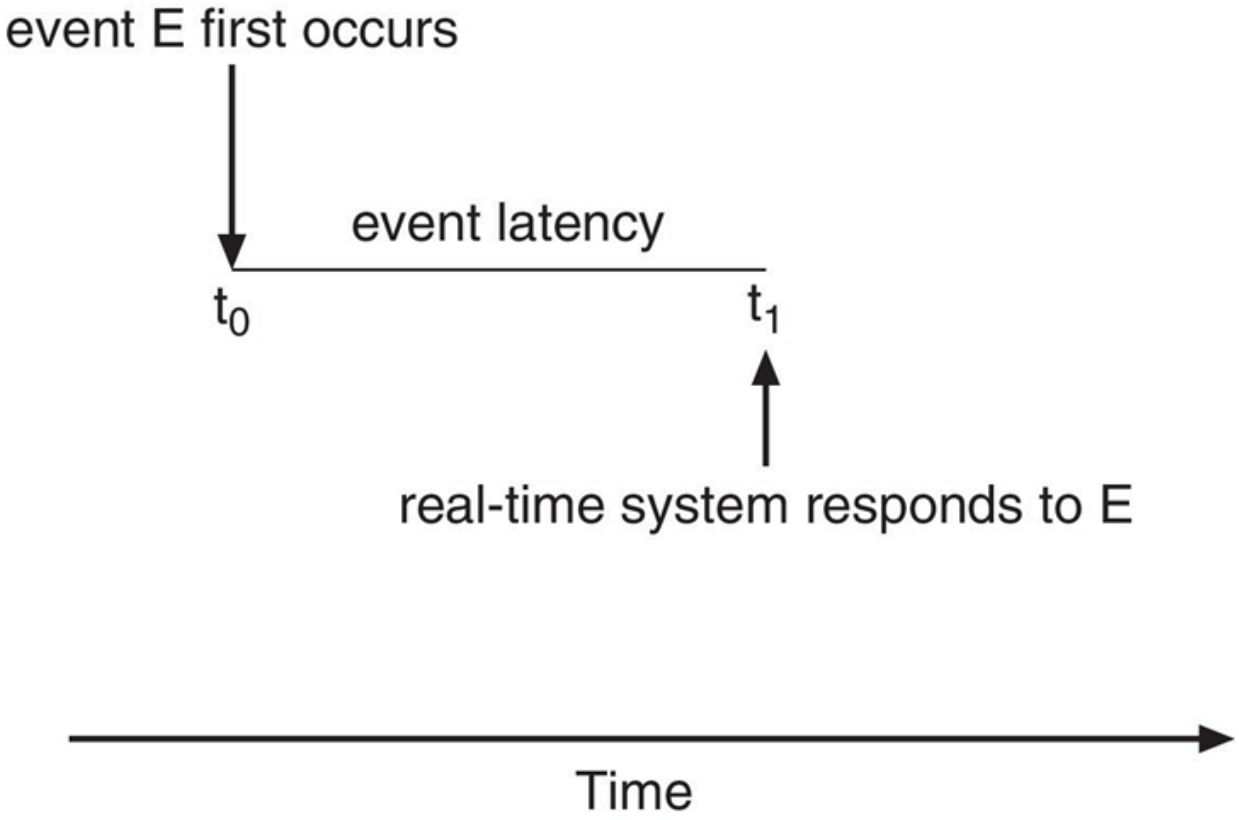
Real-Time CPU Scheduling

- Soft real-time systems: Critical real-time tasks have the highest priority but no guarantee as to when tasks will be scheduled.
- Hard real-time systems: Task must be serviced by its deadline.

Event latency is the amount of time that elapses from when an event occurs to when it is serviced. Two types of latencies affect performance

- Interrupt latency: time from arrival of interrupt to start of routine that services interrupt.

- Dispatch latency: time to switch processes.
  - Preemption of any process running in kernel mode
  - Release by low-priority process of resources needed by high-priority processes



Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive and priority-based scheduling. (Only guarantees soft real-time. Ability to meet deadlines are needed for had real-time)
- Periodic process requires CPU at constant intervals with processing time t, deadline d and period p and $0 \leq t \leq d \leq p$. Rate of period task is $\frac{1}{p}$.

Rate Monotonic Scheduling(RMS)

A priority is assigned based on the inverse of period(Higher priority for shorter periods). However, might miss the deadline when high overloading.

Earliest Deadline First Scheduling(EDF)

Priorities are assigned according to deadlines(Higher priority for earlier deadline)

Proportional Share Scheduling

Divide the CPU time to the processes according to shares.(T shares for all processes, one process may receive N shares)

POSIX Real-Time Scheduling

API provides functions for managing real-time threads

- `SCHED_FIFO` : FCFS strategy without time slices
- `SCHED_RR` : FIFO with time slices

Operating System Examples

- Linux Scheduling
  - Before Version 2.5
    - Preemptive, priority based
    - Poor response times for interactive processes
  - In Version 2.6.23+
    - Based on proportion of CPU time rather than fixed time allotments
    - Default and real-time scheduling are all included
    - CFS scheduler ensures lower priority for higher decay rate of virtual run time
  - Support load balancing but is also NUMA-aware
- Windows Scheduling
  - Preemptive, priority based
  - 32 level priority scheme, variable class is 1-15, real-time class is 16-31, 0 for memory management
  - Queue for each priority
  - For priority, Windows combines priority class and relative priority to give numeric priority.
  - Windows 7 adds user-mode scheduling(UMS), allowing applications create and manage threads independent of kernel
- Solaris
  - Priority-based scheduling and six classes available: Time sharing(default)(TS), Interactive(IA), Real time(RT), System(SYS), Fair share (FSS), Fixed priority(FP)
  - Each class has its own scheduling algorithm
  - Scheduler converts class-specific priorities into a per-thread global priority.

Algorithm Evaluation

Deterministic Modeling:

- Type of analytic evaluation
- Take a particular predetermined workload and defines the performance of each algorithm for that workload.

Deterministic Evaluation

- Calculate minimum waiting time

Queueing Models

Describes the arrival of processes and CPU and I/O bursts probabilistically. Compute average throughput, utilization, waiting time, etc.

Little's Formula

Little's law: in steady state, processes leaving queue must equal processes arriving, thus:

$$n = \lambda$$

where n is average queue length, λ is average arrival rate into queue, is average waiting time in queue.

Simulations

- Simulations are more accurate than Queueing model
- Programmed model of computer system. Clock is a variable. Gather statistics indicating algorithm performance.
- Data gathered via random number and trace tapes record sequence of real events

Implementation

Implementation have better performance than simulation but high cost and high risk.